CHAPTER 4: CONTROL STRUCTURES

PART 2



SELECTION: `IF` AND `IF...ELSE`

- One-Way Selection
- Two-Way Selection
- Compound (Block of) Statements
- Multiple Selections: Nested `if`
- Comparing `if...else` Statements with a Series of `if` Statements

SELECTION: `IF` AND `IF...ELSE` (CONTINUED)

- Using Pseudocode to Develop, Test, and Debug a Program
- Input Failure and the `if` Statement
- Confusion Between the Equality Operator (==) and the Assignment Operator (=)
- Conditional Operator (?:)

ONE-WAY SELECTION

- The syntax of one-way selection is:

```
if (expression)
    statement
```

- The statement is executed if the value of the expression is `true`
- The statement is bypassed if the value is `false`; program goes to the next statement
- `if` is a reserved word

ONE-WAY SELECTION (CONTINUED)

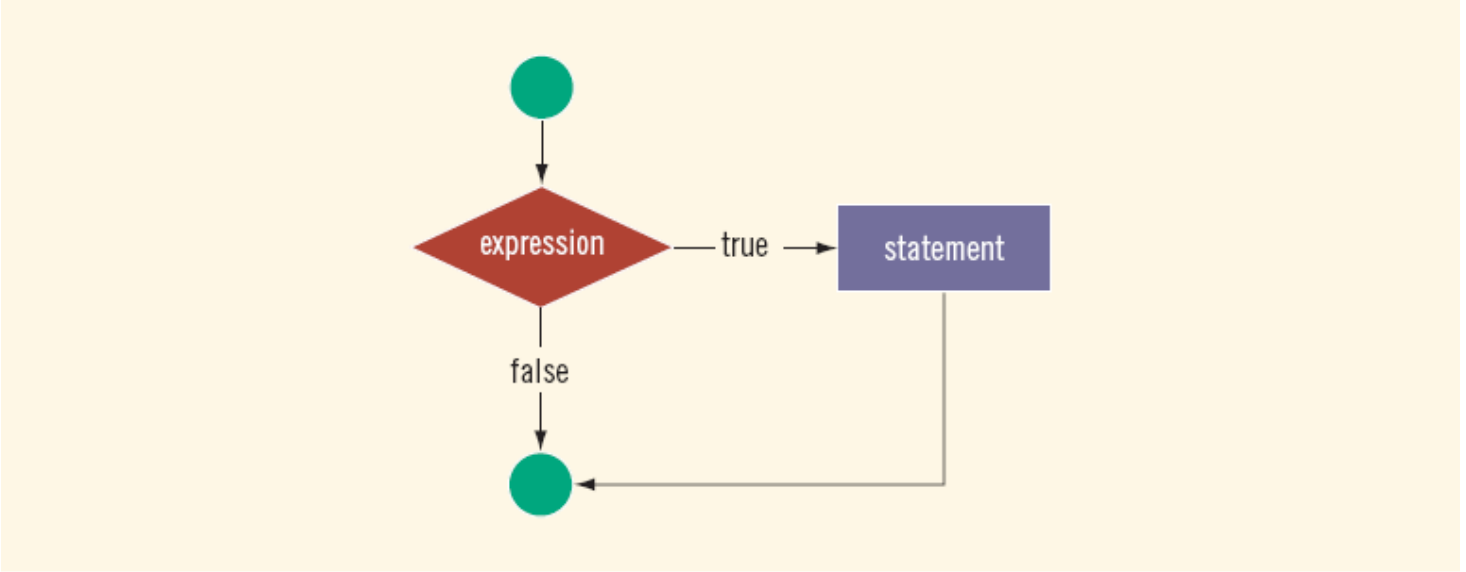


FIGURE 4-2 One-way selection

ONE-WAY SELECTION (CONTINUED)

EXAMPLE 4-9

```
if (score >= 60)
    grade = 'P';
```

In this code, if the expression (`score >= 60`) evaluates to `true`, the assignment statement, `grade = 'P';`, executes. If the expression evaluates to `false`, the statements (if any) following the `if` structure execute. For example, if the value of `score` is `65`, the value assigned to the variable `grade` is `'P'`.

EXAMPLE 4-10

The following C++ program finds the absolute value of an integer:

```
//Program: Absolute value of an integer

#include <iostream>

using namespace std;

int main()
{
    int number, temp;

    cout << "Line 1: Enter an integer: ";          //Line 1
    cin >> number;                                 //Line 2
    cout << endl;                                  //Line 3

    temp = number;                                 //Line 4

    if (number < 0)                                //Line 5
        number = -number;                          //Line 6

    cout << "Line 7: The absolute value of "
         << temp << " is " << number << endl;      //Line 7

    return 0;
}
```

Sample Run: In this sample run, the user input is shaded.

```
Line 1: Enter an integer: -6734
Line 7: The absolute value of -6734 is 6734
```

ONE-WAY SELECTION (CONTINUED)

EXAMPLE 4-11

Consider the following statement:

```
if score >= 60      //syntax error
   grade = 'P';
```

This statement illustrates an incorrect version of an `if` statement. The parentheses around the logical expression are missing, which is a syntax error.

EXAMPLE 4-12

Consider the following C++ statements:

```
if (score >= 60);          //Line 1
    grade = 'P';           //Line 2
```

Because there is a semicolon at the end of the expression (see Line 1), the `if` statement in Line 1 terminates. The action of this `if` statement is null, and the statement in Line 2 is not part of the `if` statement in Line 1. Hence, the statement in Line 2 executes regardless of how the `if` statement evaluates.

TWO-WAY SELECTION

- Two-way selection takes the form:

```
if (expression)
    statement1
else
    statement2
```

- If expression is `true`, `statement1` is executed; otherwise, `statement2` is executed
  - `statement1` and `statement2` are any C++ statements
- `else` is a reserved word

TWO-WAY SELECTION (CONTINUED)

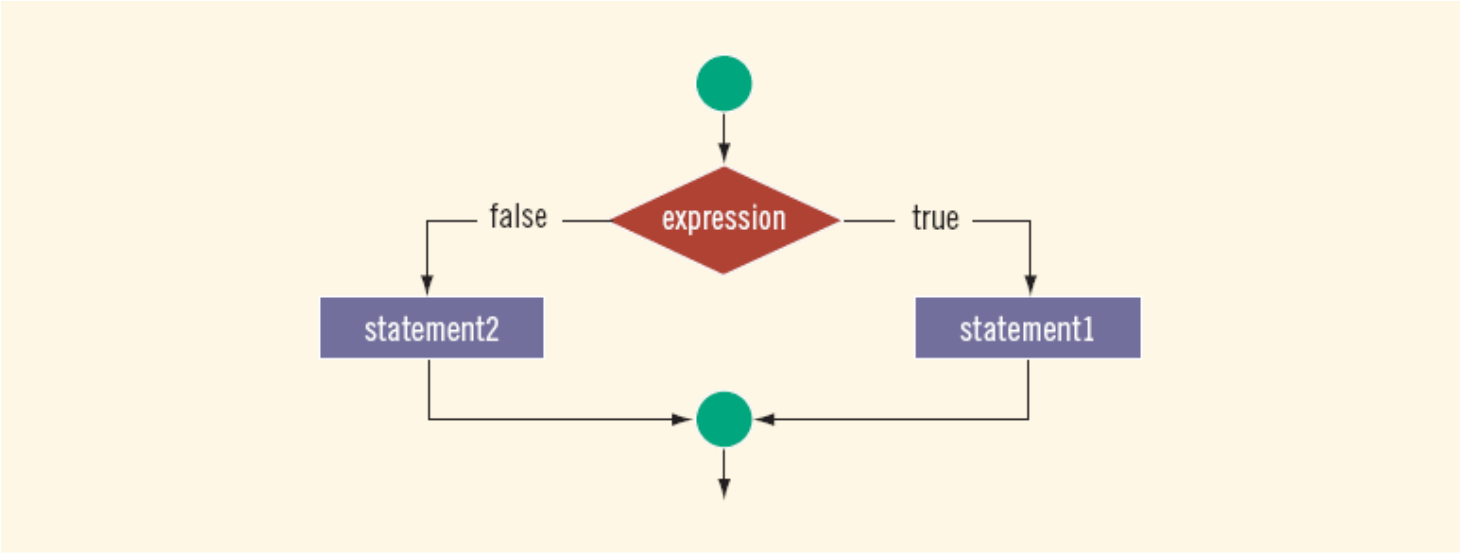


FIGURE 4-3 Two-way selection

TWO-WAY SELECTION (CONTINUED)

EXAMPLE 4-13

Consider the following statements:

```
if (hours > 40.0)                                    //Line 1
    wages = 40.0 * rate +
            1.5 * rate * (hours - 40.0);             //Line 2
else                                                 //Line 3
    wages = hours * rate;                            //Line 4
```

If the value of the variable `hours` is greater than `40.0`, then the `wages` include overtime payment. Suppose that `hours` is `50`. The expression in the `if` statement, in Line 1, evaluates to `true`, so the statement in Line 2 executes. On the other hand, if `hours` is `30`, or any number less than or equal to `40`, the expression in the `if` statement, in Line 1, evaluates to `false`. In this case, the program skips the statement in Line 2 and executes the statement in Line 4—that is, the statement following the reserved word `else` executes.

TWO-WAY SELECTION (CONTINUED)

EXAMPLE 4-14

The following statements show an example of a syntax error:

```
if (hours > 40.0);                                  //Line 1
    wages = 40.0 * rate +
            1.5 * rate * (hours - 40.0);  //Line 2
else                                                //Line 3
    wages = hours * rate;                           //Line 4
```

The semicolon at the end of the `if` statement (see Line 1) ends the `if` statement, so the statement in Line 2 separates the `else` clause from the `if` statement. That is, `else` is all by itself. Because there is no stand-alone `else` statement in C++, this code generates a syntax error.

COMPOUND (BLOCK OF) STATEMENT

- Compound statement (block of statements):

```
{
    statement1
    statement2
       .
       .
       .
    statementn
}
```

- A compound statement is a single statement

COMPOUND (BLOCK OF) STATEMENT (CONTINUED)

```
if (ag e > 18)
{
   cout << "Eligible to vote." << endl;
   cout << "No longer a minor." << endl;
}
else
{
   cout << "Not eligible to vote." << endl;
   cout << "Still a minor." << endl;
}
```

MULTIPLE SELECTIONS: NESTED `IF`

- Nesting: one control statement in another
- An `else` is associated with the most recent
  `if` that has not been paired with an `else`

EXAMPLE 4-18

Suppose that `balance` and `interestRate` are variables of type `double`. The following statements determine the `interestRate` depending on the value of the `balance`:

```
if (balance > 50000.00)                    //Line 1
    interestRate = 0.07;                   //Line 2
else                                       //Line 3
    if (balance >= 25000.00)               //Line 4
        interestRate = 0.05;               //Line 5
    else                                   //Line 6
        if (balance >= 1000.00)            //Line 7
            interestRate = 0.03;           //Line 8
        else                               //Line 9
            interestRate = 0.00;           //Line 10
```

To avoid excessive indentation, the code in Example 4-18 can be rewritten as follows:

```
if (balance > 50000.00)                    //Line 1
    interestRate = 0.07;                   //Line 2
else if (balance >= 25000.00)              //Line 3
    interestRate = 0.05;                   //Line 4
else if (balance >= 1000.00)               //Line 5
    interestRate = 0.03;                   //Line 6
else                                       //Line 7
    interestRate = 0.00;                   //Line 8
```

COMPARING IF...ELSE STATEMENTS WITH A SERIES OF IF STATEMENTS

a.
```
if (month == 1)                          //Line 1
    cout << "January" << endl;           //Line 2
else if (month == 2)                     //Line 3
    cout << "February" << endl;          //Line 4
else if (month == 3)                     //Line 5
    cout << "March" << endl;             //Line 6
else if (month == 4)                     //Line 7
    cout << "April" << endl;             //Line 8
else if (month == 5)                     //Line 9
    cout << "May" << endl;               //Line 10
else if (month == 6)                     //Line 11
    cout << "June" << endl;              //Line 12
```

b.
```
if (month == 1)
    cout << "January" << endl;
if (month == 2)
    cout << "February" << endl;
if (month == 3)
    cout << "March" << endl;
if (month == 4)
    cout << "April" << endl;
if (month == 5)
    cout << "May" << endl;
if (month == 6)
    cout << "June" << endl;
```

USING PSEUDOCODE TO DEVELOP, TEST, AND DEBUG A PROGRAM

- Pseudocode (pseudo): provides a useful means to outline and refine a program before putting it into formal C++ code
- You must first develop a program using paper and pencil
- On paper, it is easier to spot errors and improve the program
  - Especially with large programs

INPUT FAILURE AND THE `IF` STATEMENT

- If input stream enters a fail state
  - All subsequent input statements associated with that stream are ignored
  - Program continues to execute
  - May produce erroneous results
- Can use `if` statements to check status of input stream
- If stream enters the fail state, include instructions that stop program execution

CONFUSION BETWEEN == AND =

- C++ allows you to use any expression that can be evaluated to either `true` or `false` as an expression in the `if` statement:

```
if (x = 5)
    cout << "The value is five." << endl;
```

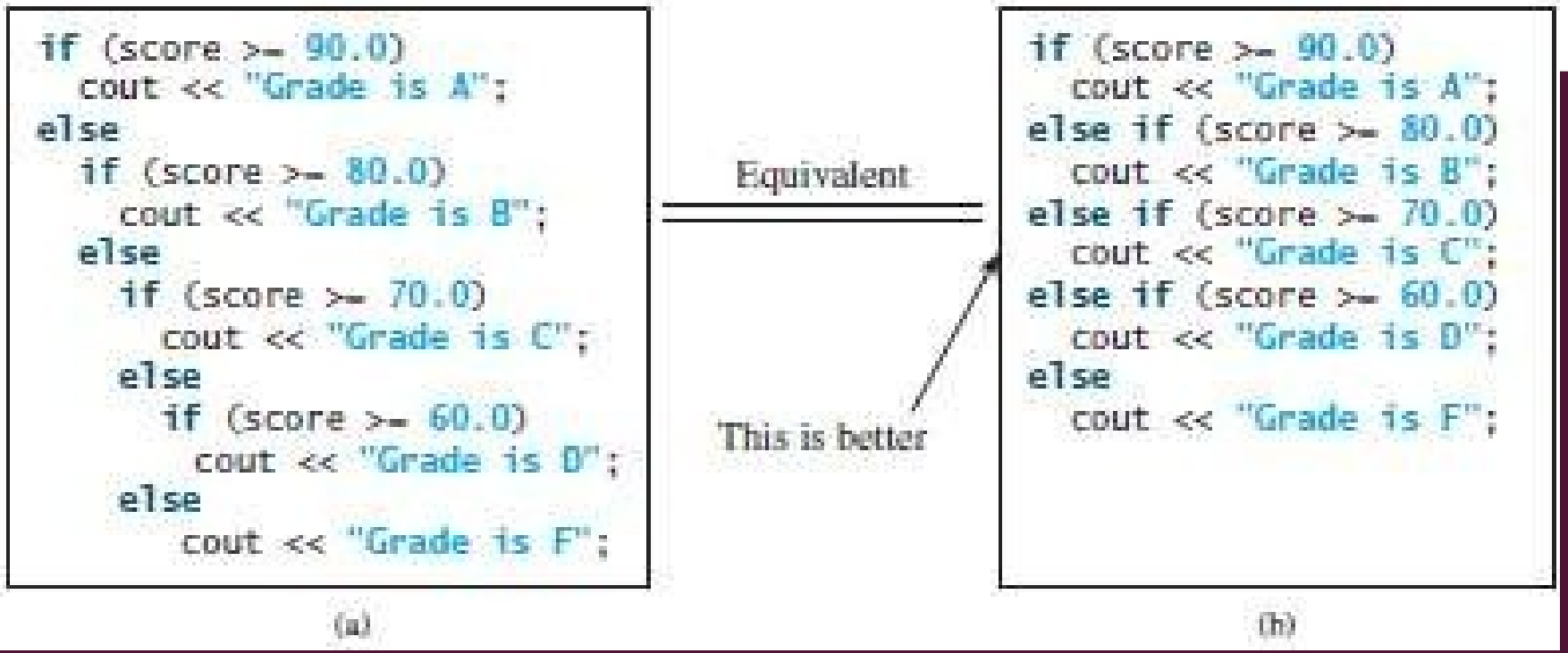
- The appearance of = in place of == resembles a *silent killer*
  - It is not a syntax error
  - It is a logical error

CONDITIONAL OPERATOR (?:)

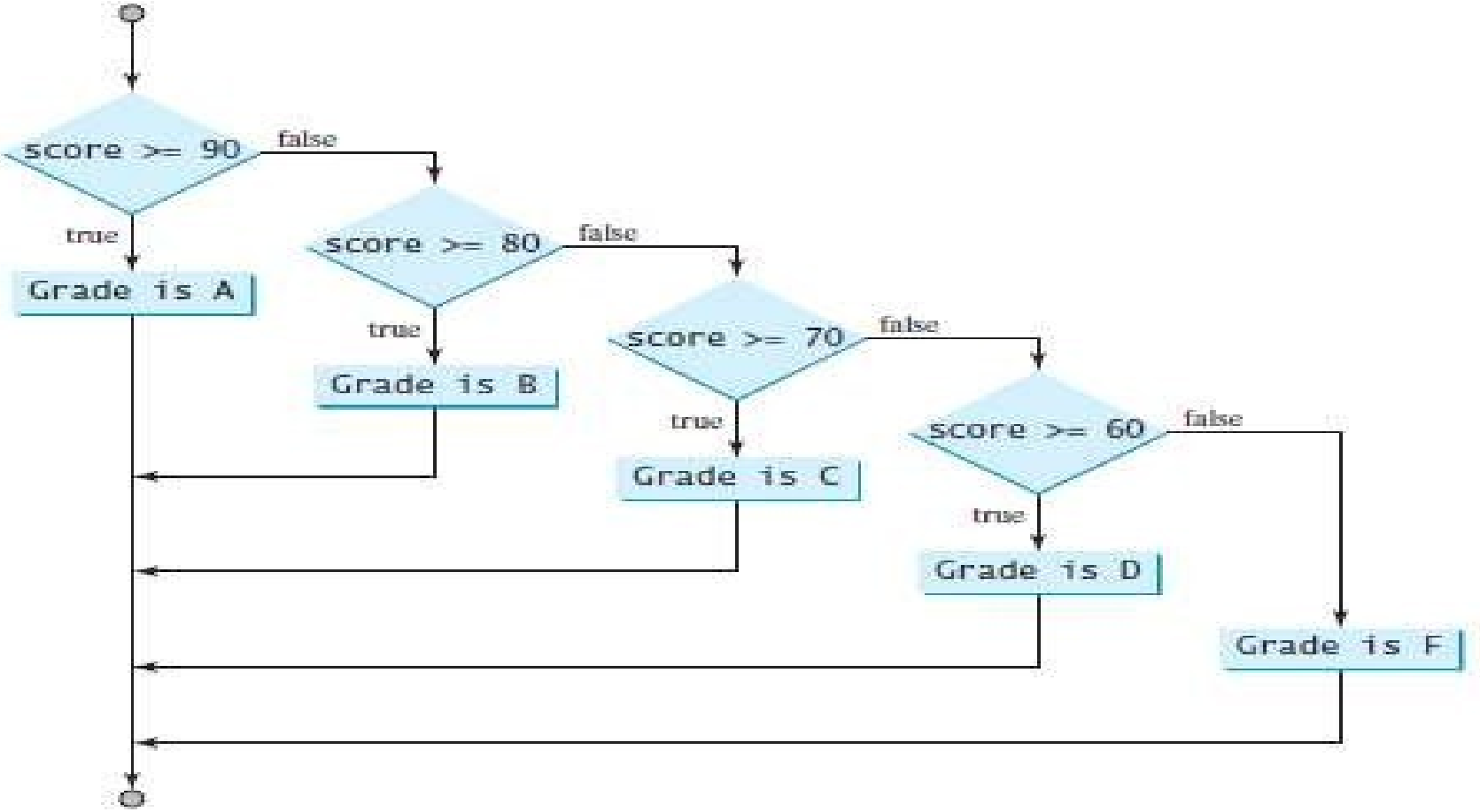
- Conditional operator (`?:`) takes three arguments
  - Ternary operator
- Syntax for using the conditional operator:
  `expression1 ? expression2 : expression3`
- If `expression1` is `true`, the result of the conditional expression is `expression2`
  - Otherwise, the result is `expression3`

MULTI-WAY IF ELSE

More than two possibilities, for example your grade in **programming fundamentals course**



MULTIPLE WAY IF



MULTIPLE WAY IF

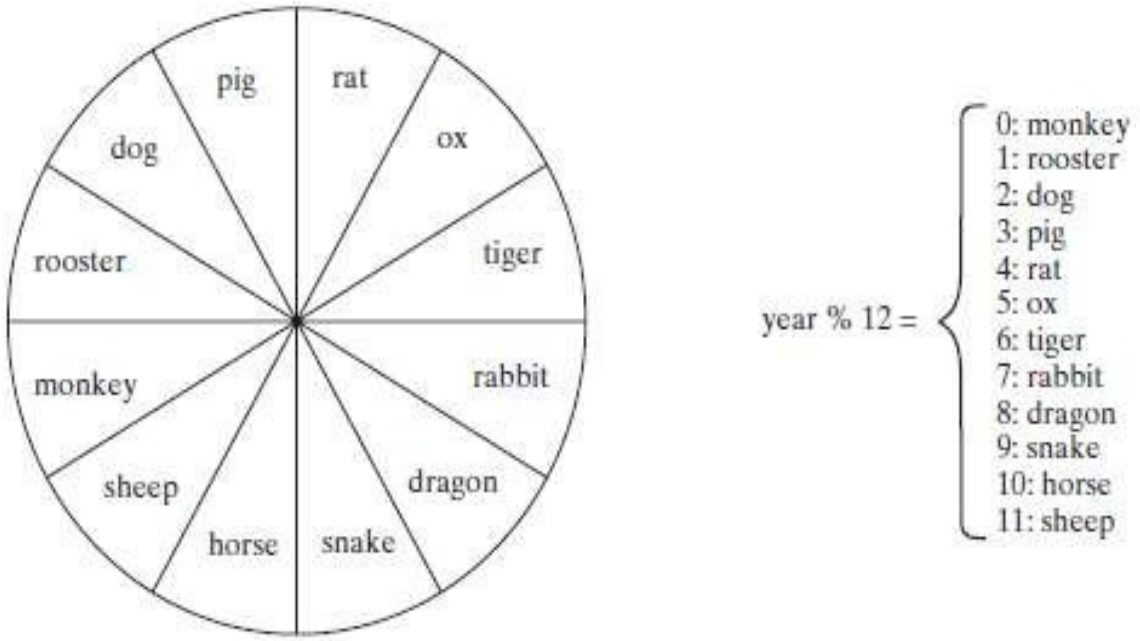


FIGURE 3.6 The Chinese Zodiac is based on a twelve-year cycle.

NESTED IF

Task 1: Write a code using nested if in which user enter three numbers, and find if 1st number is greater or not.

```
int num1,num2,num3
cin>>num1>>num2>>num3;
        if(num1>num2)
        {
                If(num2>num3)
                {
                cout<<num1<<“ is greater”
                }
                else
                {
                cout<<num1<<“ is not greater”;
                }
        }
        else
        {
        cout<<num1<<“ is not greater”; }
```

LOGICAL OPERATORS

- *The logical operators !, &&, and || can be used to create a compound Boolean expression*

TABLE 3.3 Boolean Operators

Operator	*Name*	*Description*
`!`	not	logical negation
`&&`	and	logical conjunction
`||`	or	logical disjunction

TABLE 3.4 Truth Table for Operator `!`

p	*!p*	*Example (assume age = 24, weight = 140)*
`true`	`false`	`!(age > 18)` is `false`, because `(age > 18)` is `true`.
`false`	`true`	`!(weight == 150)` is `true`, because `(weight == 150)` is `false`.

LOGICAL OPERATORS

TABLE 3.5 Truth Table for Operator &&

p1	*p2*	*p1 && p2*	*Example (assume age = 24, weight = 140)*
`false`	`false`	`false`	`(age > 18) && (weight <= 140)` is `true`, because
`false`	`true`	`false`	`(age > 18)` and `(weight <= 140)` are both `true`.
`true`	`false`	`false`	`(age > 18) && (weight > 140)` is `false`, because
`true`	`true`	`true`	`(weight > 140)` is `false`.

TABLE 3.6 Truth Table for Operator ||

p1	*p2*	*p1 || p2*	*Example (assume age = 24, weight = 140)*
`false`	`false`	`false`	`(age > 34) || (weight <= 140)` is `true`, because
`false`	`true`	`true`	`(weight <= 140)` is `true`.
`true`	`false`	`true`	`(age > 34) || (weight >= 150)` is `false`, because
`true`	`true`	`true`	`(age > 34)` and `(weight >= 150)` are both `false`.

PROGRAM THAT CHECKS WHETHER A NUMBER IS DIVISIBLE BY2 AND 3, BY 2 OR 3, AND BY 2 OR 3 BUT NOT BOTH:

```
#include <iostream>
using namespace std;

int main()
{
  int number;
  cout << "Enter an integer: ";
  cin >> number;

  if (number % 2 == 0 && number % 3 == 0)
    cout << number << " is divisible by 2 and 3." << endl;

  if (number % 2 == 0 || number % 3 == 0)
    cout << number << " is divisible by 2 or 3." << endl;

  if ((number % 2 == 0 || number % 3 == 0) &&
      !(number % 2 == 0 && number % 3 == 0))
    cout << number << " divisible by 2 or 3, but not both." << endl;

  return 0;
}
```

```
Enter an integer: 4
4 is divisible by 2 or 3.
4 is divisible by 2 or 3, but not both.
```

```
Enter an integer: 18
18 is divisible by 2 and 3
18 is divisible by 2 or 3.
```

TASKS

- Write a Boolean expression that evaluates to true if age is greater than 13 and less than 18.
- Write a Boolean expression that evaluates to true if weight is greater than 50 pounds or height is greater than 60 inches.
- Write a Boolean expression that evaluates to true if weight is greater than 50 pounds and height is greater than 60 inches
- Write a Boolean expression that evaluates to true if either weight is greater than 50 pounds or height is greater than 60 inches, but not both.

Combine all to make a program that takes input from user then checks all expressions separately and gives output accordingly.

TASKS